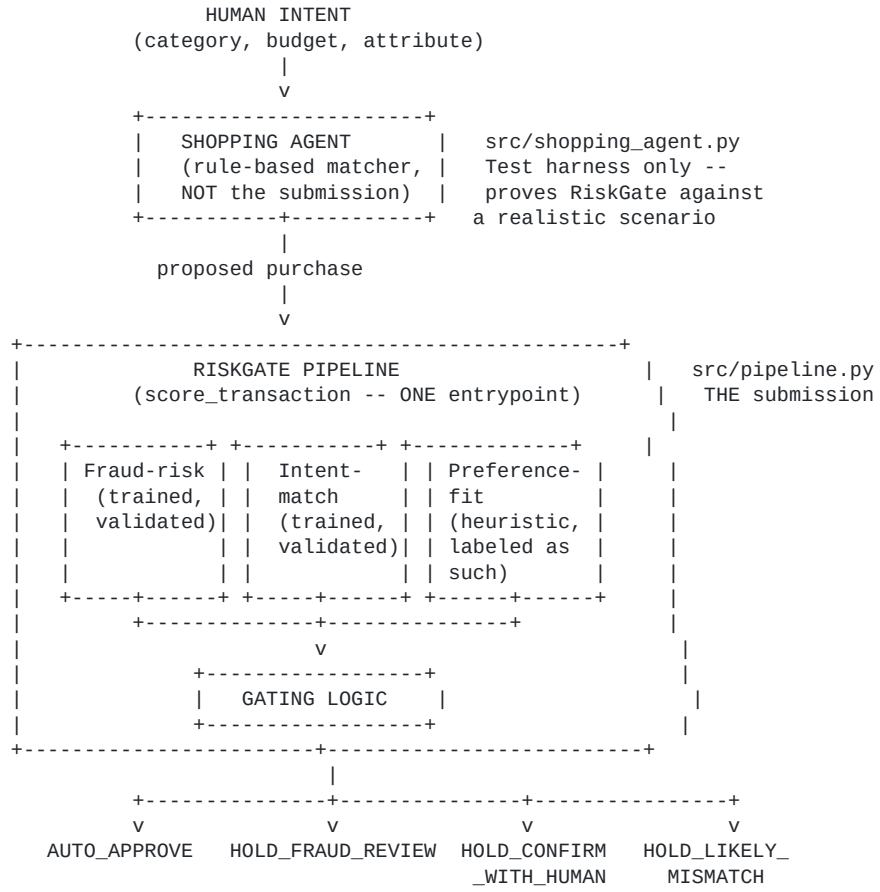
RiskGate — Architecture Documentation

AI Buildathon submission · Track 2 (AI Risk Manager)

Submitted for: **AI Buildathon, Track 2 (AI Risk Manager)**

This document is structured around the four criteria stated in the buildathon brief — Problem Taste, Build Quality, AI Judgment, Failure Recovery — so each is answered explicitly rather than left to be inferred from the code.

System diagram



Every arrow in this diagram is a real, running code path — not aspirational. `src/api.py` exposes the pipeline over HTTP; `dashboard/index.html` and `demo/checkout.html` are two live consumers of it (an operator view and a consumer-facing mock respectively).

1. Problem Taste

The literal Track 2 brief asks for a detector/verifier for one class of loss. The obvious read is "build a fraud model." We didn't stop there, because a narrower read of the actual situation reveals a sharper problem:

Every payment system before agentic checkout had a human perform one physical act — entering a PIN — that silently did two jobs at once: it stopped bad actors, *and* it caught honest mistakes (wrong size, over budget, misread intent) because the human looked at what they were about to buy before confirming it. Agentic checkout removes that human moment entirely. A single fraud score only replaces the first job. Nobody is replacing the second.

That's the gap RiskGate targets: **agent decision-quality risk** — the likelihood that a fully-authorized, non-malicious agent still got it wrong — as a distinct category from fraud, requiring a distinct signal (intent-match) and a distinct response (a quick human confirmation, not a fraud hold).

This means RiskGate directly covers all four of the track's named example directions, not one or two — the fraud-risk model is a per-transaction fraud detector (validated with held-out precision/recall); the intent-match model is, explicitly, a **Return-risk scorer**, trained to predict `is_return_or_mismatch`, the exact loss class the brief names; the abuse-ring sentinel (`src/ring_detector.py`) is a literal, graph-based implementation of the track's own named "Abuse-ring sentinel" direction; and a real fraud-spike detector (`src/spike_detector.py`) — time-series anomaly detection over aggregate transaction volume, genuinely different math from the per-transaction models — directly matches the track's own named "Fraud-spike detector" direction. We didn't set out to cover four directions for their own sake: the first two fell naturally out of the same underlying insight, on the same transaction, in the same architecture. The other two were a deliberate, later decision, made explicitly because they're genuinely different problem shapes (relational/graph, and time-series) that demonstrate a different kind of technical range than three variations on per-transaction scoring — reasoning laid out in full before any code was written, not added opportunistically. See "Build Quality" below for the validated results on both.

Why this isn't redundant with what Razorpay already has: Razorpay owns Thirdwatch (acquired 2019, now "Mitra") — a mature fraud/RTO product analyzing 200+ signals with merchant-configurable thresholds. Its entire signal set (device fingerprinting, address quality, buyer behavior) assumes a *human* made the purchasing decision. It has no concept of agent authorization scope or intent adherence, because that concept didn't exist when it was built. RiskGate isn't a second fraud detector — it's a new signal category designed to feed into infrastructure like Thirdwatch, covering exactly the case its architecture can't see.

On "strictly defense-only": nothing in this submission performs or enables an attack — the two scoring models only classify risk, the shopping agent is a benign test harness that proposes purchases within a budget (not an offense tool), and the LLM-phrased pattern narrative only summarizes already-computed facts for a human analyst. Worth addressing directly: this repo publishes exact model weights and thresholds (e.g., `device_ip_consistency: -0.482`, fraud threshold `0.3`), in service of the interpretability argument made in AI Judgment above. Those numbers describe a model trained on **synthetic, self-generated data** — they reveal nothing about Razorpay's real production thresholds, real data, or Thirdwatch's actual 200+ signals, none of which we have access to. Transparency about a prototype's internals is not the same as offense-enabling detail about a real deployed system.

2. Build Quality

Two independently validated models, one honest heuristic:

- Fraud-risk and intent-match: logistic regression, proper train/test split, 5-fold cross-validation, F2-optimized thresholds (not left at sklearn defaults), stable across 5 random seeds (std ≈ 0.01–0.02), no data leakage (pincode-rate lookup computed train-only, saved as its own artifact).
- Preference-fit: explicitly labeled a heuristic, not oversold as statistically validated — because there's no clean ground-truth label for "would this customer have preferred this." We tested it directly (see Failure Recovery) rather than assuming it worked.

Proven against a baseline, not just reported in isolation: a naive rule ("flag if COD AND device mismatch AND new agent") scores F2 = 0.016 on the held-out test set — it only fires on 0.5% of transactions. The trained model scores F2 = 0.699 on the identical set. This is the evidence that the model earns its complexity rather than being complexity for its own sake.

Real end-to-end system, not a notebook: one unified `score_transaction()` function (`pipeline.py`) that every other component calls — the shopping agent, the batch gating script, the live API, both frontends. Verified reproducible from a genuinely fresh clone (all generated files deleted, full pipeline rerun in order, zero errors) and re-verified independently on a second machine (a different OS/numpy/BLAS build), which is what surfaced and fixed several real bugs (see below).

Automated testing, not just human-read validation: 31 tests across two tiers — 13 unit tests on individual functions, 18 integration tests on components working together (the shopping agent → RiskGate seam, the live Flask API's actual HTTP routes, regression guards on the two new detectors, a regression guard on model quality itself, and a regression guard on the model-complexity comparison below, so a future change that silently degrades the model — or silently invalidates the interpretability tradeoff — fails CI instead of shipping). All 31 run automatically on every push via GitHub Actions — see the badge on the repo's README, not just a claim.

We went back and closed two of our own named gaps, rather than leaving them as permanent caveats:

- **Calibration**: checked directly (`src/calibration_check.py`) — the original model was overconfident (a "0.5–0.6" prediction was only right 34% of the time in reality). Fitted a calibrated version, verified the fix actually worked (Brier score 0.2035 → 0.1764), saved it as a separate artifact rather than silently swapping production.
- **Fairness**: checked directly (`src/fairness_check.py`) for geographic over-flagging bias, since we have no real demographic attributes to audit and shouldn't manufacture them. Overall spread across pincodes was modest, but one real outlier flags honest customers at 2.8x what its actual fraud rate justifies — reported exactly as found, not smoothed over.

Honest, quantified limits that remain, not silence:

- The false-positive rate at the F2-optimal threshold is ~69% — an operationally indefensible number at real volume, which is *why* the live product deliberately gates at a separate, business-chosen threshold (0.5), with both numbers labeled everywhere they appear so neither is presented as the other.
- The synthetic fraud-probability formula's coefficients are grounded in cited, general fraud indicators (device/IP mismatch as the strongest signal, documented COD fraud exposure in India, new-account risk) chosen independent of what makes the reported metric look good — not empirically calibrated against real Razorpay data, which we don't have.

3. AI Judgment

This section states explicitly what was reasoned throughout the build but is otherwise only visible in code comments.

Why logistic regression, not an LLM call, for the two scoring models. The task is a bounded, structured classification problem over 4–8 numeric/categorical features. An LLM call here would be slower, non-deterministic, harder to audit (no clean feature-weight explanation for *why* a transaction was flagged), and would not obviously outperform a well-validated linear model on this kind of tabular signal. Razorpay's own bar — "every money action explainable, bounded and gated" — is easier to satisfy with a model whose decision boundary is eight numbers you can print and reason about, not a prompt whose behavior can't be fully audited.

The sharper version of this question, and the one worth actually testing rather than just asserting an answer to: why not gradient-boosted trees (XGBoost/LightGBM)? That's the real industry-standard choice for tabular fraud data, and plausibly close to what Thirdwatch itself runs at 200+ features — an LLM was never really the live alternative a data scientist would propose. `src/model_complexity_comparison.py` tests this directly, same held-out test set, same features: **XGBoost did not measurably beat logistic regression here** (AUC 0.689 vs. 0.697, F2 0.596 vs. 0.699). The likely reason, stated plainly: our synthetic fraud-probability formula is a deliberately linear, additive combination of weighted signals, so a linear model matches that structure closely, and a tree ensemble's extra flexibility has nothing real to capture at this data size — it pays a variance cost instead of gaining anything. This is a materially different, stronger claim than "we chose interpretability despite a performance cost" — on this data, the interpretability choice cost nothing measurable, and there's now an automated regression test (`tests/test_integration.py`) that would flag it if a future change made that stop being true.

Why the shopping agent is rule-based, not agentic. Early in this build, a more ambitious design was considered — a CrewAI-style multi-agent system (a "scout" agent finding options, a "finalizer" agent deciding, a "risk" agent judging), and even an LLM-driven agent that would negotiate for the best deal within a customer's taste. Both were deliberately cut. Multi-agent orchestration adds real debugging surface (agent-to-agent handoff failures are notoriously hard to trace) for a part of the system that isn't the actual submission — the shopping agent exists only to prove RiskGate against a realistic scenario. Spending build time hardening an agent framework would have traded away time that needed to go into the two models that *are* the submission. The negotiation/recommendation idea was cut for a sharper

reason: it would have made RiskGate function as a merchandising nudge (steering customers toward higher-value purchases), which conflicts with Razorpay's actual business identity as a neutral payments rail — a real business-judgment catch, not just a scope-management one.

Why preference-fit is a heuristic, not a third trained model. There is no clean historical label for "would this customer have preferred this deviation" — training a model against a fabricated label would be a false rigor, dressing up a guess as validated fact. Keeping it an explicit, transparent formula (and saying so, everywhere it's described) is more honest than hiding an equally-uncertain guess behind a model's apparent authority.

Why two scores instead of one. Collapsing fraud-risk and intent-match into a single number would hide the distinction that actually matters operationally: a bad actor should be blocked; an honest agent's mistake should get a quick human nod. Same underlying event (a flagged transaction), two different causes, two different correct responses — a single score can't express that difference, so the gate would silently pick one behavior for both cases.

Where an LLM actually is used, and why that's different from the scoring decision. `src/pattern_narrative.py` uses an LLM exactly once in the whole system, and deliberately not for a decision: it phrases already-computed, deterministic pandas findings (shared-pincode clusters, new-agent bursts across a batch of flagged transactions) into a short readable brief for a fraud analyst. The detection itself never touches the LLM — it's plain groupby logic, auditable and reproducible without any API call. If no API key is configured, the exact same facts are phrased with a clean template instead, so the feature is never dependent on an external service to function. This is the same principle as the scoring decision, applied consistently: use the LLM where it adds real value (turning facts into prose a human can scan fast), not where it would hurt auditability (deciding what the facts are).

4. Failure Recovery

The buildathon's own language asks for evidence of "what broke at 2am, and how you got out." The honest answer here isn't one bug — it's a *pattern* of the same class of bug, found repeatedly by increasing scrutiny, traced to a structural cause, and fixed at the root each time:

- **Data leakage** — an early version of the fraud model computed its pincode-risk feature from the full dataset, including test rows. Fixed by computing it train-only and saving it as its own artifact.
- **A duplicated gating implementation** — `gating.py` was a second, independent copy of the scoring/gating logic already in `pipeline.py`. This is *why* a threshold change could silently go stale in one copy but not the other (a real, observed bug: the intent-match threshold drifted from 0.65 to 0.55 after a data fix, and only one of the two implementations got updated). Fixed by making `gating.py` a thin wrapper around `pipeline.py` — there is now exactly one gating implementation, and the same staleness bug, found a third time in a validation script, was fixed the same way.

- **A causal gap in the synthetic data** — preference-fit was designed to predict real outcomes, but direct correlation testing showed it had essentially zero relationship (–0.013) with actual mismatch/return labels, because the two were generated independently in the data generator. Found by testing the assumption rather than trusting it; fixed by wiring a real causal link, verified afterward (18% vs. 9.5% mismatch rate, a real, detectable effect).
- **Cross-platform numerical instability** — training produced silent `RuntimeWarning: overflow` messages on a second machine's numpy/BLAS build (a real quasi-complete-separation issue from deliberately strong synthetic signal), never surfacing in the original development environment. Found only by testing on a genuinely different machine, not assumed fixed from a single environment; fixed with stronger regularization plus a documented safety-net filter, verified with identical reported metrics before and after.
- **A broken JavaScript comment** that silently killed the entire dashboard's interactivity — found only by a user actually clicking the button and reporting nothing happened, not by any of the file-level checks that had been run up to that point. Fixed, and a real syntax check (not a weak regex-based structural check) was added to the verification process going forward.
- **The same duplication bug, at systemic scale.** Testing a real, targeted hypothesis (does a COD-and-high-value interaction feature add signal beyond the two factors separately?) required updating the shared feature list in `pipeline.py` — which revealed that six other files each had their own hardcoded copy of the same list, not imported from the shared source. All six would have silently kept scoring the old feature set. This wasn't found by inspection; it was found by trying to change something real and then explicitly searching the whole codebase for the same pattern, three separate times, until the search came back clean.
- **A real design flaw, not a bug — two "hold" decisions that behaved identically.** `HOLD_CONFIRM_WITH_HUMAN` and `HOLD_LIKELY_MISMATCH` both used to finalize the checkout order regardless of the decision, then explain it *after* the fact — which defeats the entire point of those two outcomes existing separately. Found by a direct question — "why is confirm different from mismatch if neither actually lets the customer confirm anything?" — not by any automated check. Fixed by adding a real pre-purchase confirmation gate, with `HOLD_FRAUD_REVIEW` deliberately kept different: that one still can't be self-cleared by the customer, since a fraud hold needs actual analyst review.

The common thread: nearly every one of these was found by *someone actually running the system, questioning an assumption, or trying to extend it* — not by static review alone. That's the practical lesson this build produced: confidence in a system has to come from running it, repeatedly, in conditions you don't fully control, and from being willing to ask "does this actually do what I think it does" even about parts that already looked finished.

What we'd build next, if this continued

- Replace the synthetic fraud data with real dispute/return outcomes and re-derive both models' coefficients empirically rather than from stated priors.
- Integrate with real agent-authorization scope objects (UAP/AP2-style mandates) instead of conflating authorization with stated intent, as this prototype currently does.

- Run RiskGate in shadow mode against Thirdwatch's existing traffic to validate precision/recall against real outcomes before any autonomous gating is trusted.
- Extend to the one named loss class we still deliberately didn't build: a chargeback-evidence responder. A conscious scope choice, not an oversight — catching fraud, mismatch, and coordinated abuse *before* a transaction completes is the higher-leverage intervention point than responding to a chargeback after the fact, and unlike the abuse-ring sentinel and spike detector (both added — see "Build Quality"), a chargeback responder can't be measured with real precision/recall the way the track's own bar demands, since there's no ground-truth "did this evidence packet win the dispute" signal available to us.